Лабораторна робота №7

«Робота з JSON та підготовка даних для відображення»

Мета

Сформувати практичні навички роботи з форматом JSON у мові Dart, навчитися зчитувати, створювати, редагувати та видаляти дані у JSON-файлі, перетворювати JSON-дані у об’єкти моделей (model class), використовувати fromJson() та toJson(), а також відображати список даних у Flutter-застосунку за допомогою ListView.builder.

Теоретичні відомості

1. Формат JSON

JSON (JavaScript Object Notation) — це текстовий формат обміну даними, який широко використовується у веб-сервісах, мобільних застосунках та API.

JSON складається з:

- об’єктів { }
- масивів [ ]
- пар ключ → значення

JSON-об’єкт містить дані у вигляді структурованих полів, де кожне поле має ключ та значення.

Основні типи даних у JSON:

- рядок (String)
- число (Number)
- логічне значення (Boolean)
- масив (Array)
- об’єкт (Object)

Приклад JSON-об’єкта:

```
{
"id": 1,
"title": "Завдання 1",
"isFavorite": true,
"status": "active"
}
```

Приклад JSON-масиву:

```
[
{
"id": 1,
"title": "Завдання 1",
"isFavorite": false,
"status": "active"
},
{
"id": 2,
"title": "Завдання 2",
"isFavorite": true,
"status": "done"
}
]
```

JSON використовується для:

- збереження локальних даних;
- обміну даними з API;
- передачі інформації між компонентами застосунку;
- підготовки даних для відображення у списках.

2. Робота з JSON у Dart

Для роботи з JSON у Dart використовується бібліотека:

```
import 'dart:convert';
```

Основні функції:

- jsonDecode() — перетворює JSON-рядок у Map або List;
- jsonEncode() — перетворює Dart-об’єкт у JSON-рядок.

Приклад декодування JSON:

```
import 'dart:convert';
void main() {
 String jsonString = '{"title": "Task", "isFavorite": false}';
 Map<String, dynamic> data = jsonDecode(jsonString);
 print(data['title']);
}
```

Приклад кодування JSON:

```
import 'dart:convert';

void main() {
 Map<String, dynamic> data = {
   'id': 1,
   'title': 'Task',
   'isFavorite': true,
 };

 String jsonString = jsonEncode(data);

 print(jsonString);
}
```

3. Зчитування JSON з файлу

У Flutter дані можна зчитувати з локального JSON-файлу.

Якщо файл розміщений, наприклад, у папці assets, його можна завантажити як текстовий рядок.

Приклад зчитування JSON-файлу з assets:

```
import 'package:flutter/services.dart' show rootBundle;

Future<String> loadJsonFile() async {
 return await rootBundle.loadString('assets/data/items.json');
}
```

Приклад отримання JSON-рядка:

```
Future<void> readData() async {
 String jsonString = await loadJsonFile();
 print(jsonString);
}
```

Щоб такий файл працював, його потрібно додати в pubspec.yaml:

```
flutter:

assets:

- assets/data/items.json
```

4.Запис JSON у файл

Після змін даних їх можна записати у файл. Для цього зазвичай використовують пакет path_provider і клас File.

Приклад запису JSON-рядка у файл:

```
import 'dart:io';
import 'package:path_provider/path_provider.dart';
```

```
Future<File> writeJsonToFile(String jsonString) async {
 final directory = await getApplicationDocumentsDirectory();
 final file = File('${directory.path}/items.json');
 return file.writeAsString(jsonString);
}
```

Приклад виклику:

```
Future<void> saveData() async {
 String jsonString = '[{"id":1,"title":"Task 1"}]';
 await writeJsonToFile(jsonString);
}
```

5. Зчитування JSON із локального файлу застосунку

Якщо файл уже був записаний у пам’ять пристрою, його можна прочитати так:

```
import 'dart:io';
import 'package:path_provider/path_provider.dart';

Future<String> readJsonFromFile() async {
 final directory = await getApplicationDocumentsDirectory();
 final file = File('${directory.path}/items.json');

 if (await file.exists()) {
   return await file.readAsString();
 } else {
   return '[]';
 }
}
```

Приклад використання:

```
Future<void> loadSavedData() async {
 String jsonString = await readJsonFromFile();
 print(jsonString);
}
```

6. Оновлення JSON після змін

Після додавання, редагування або видалення даних потрібно оновити JSON-представлення списку:

```
String saveItemsToJson() {
 return jsonEncode(
   items.map((item) => item.toJson()).toList(),
 );
}
```

Цей метод можна викликати після кожної зміни списку.

7. Метод fromJson()

Метод fromJson() використовується для створення об’єкта моделі з JSON-даних.

```
class ItemModel {
 final int id;
 String title;
 bool isFavorite;
 ItemStatus status;

 ItemModel({
   required this.id,
   required this.title,
   this.isFavorite = false,
   this.status = ItemStatus.active,
 });

 factory ItemModel.fromJson(Map<String, dynamic> json) {
   return ItemModel(
     id: json['id'],
     title: json['title'],
     isFavorite: json['isFavorite'],
     status: json['status'] == 'done'
         ? ItemStatus.done
         : ItemStatus.active,
```

```
        );
      }
    }
```

Приклад використання:

```
Map<String, dynamic> data = {
 'id': 1,
 'title': 'Task 1',
 'isFavorite': true,
 'status': 'active',
};

ItemModel item = ItemModel.fromJson(data);
print(item.title);
```

8. Метод toJson()

Метод toJson() виконує зворотну операцію — перетворює модель у JSON-структуру.

Приклад:

```
class ItemModel {
 final int id;
 String title;
 bool isFavorite;
 ItemStatus status;

 ItemModel({
   required this.id,
   required this.title,
   this.isFavorite = false,
   this.status = ItemStatus.active,
 });

 Map<String, dynamic> toJson() {
   return {
     'id': id,
     'title': title,
     'isFavorite': isFavorite,
```

```
      'status': status.name,
    };
  }
}
```

Приклад використання:

```
ItemModel item = ItemModel(
 id: 1,
 title: 'Task 1',
 isFavorite: true,
 status: ItemStatus.done,
);

Map<String, dynamic> jsonMap = item.toJson();
print(jsonMap);
```

9. Перетворення JSON-масиву у список моделей

JSON часто містить масив об’єктів, який потрібно перетворити у список моделей.

Для цього:

1. JSON декодується у список (List<dynamic>).
2. Кожен елемент перетворюється у модель через fromJson().
3. Створюється список типу List<Model>.

Приклад JSON:

```
{"id": 1, "title": "Task 1", "isFavorite": false, "status": "active"},
{"id": 2, "title": "Task 2", "isFavorite": true, "status": "done"}
```

Приклад перетворення у список моделей:

```
import 'dart:convert';

String jsonString = '''
[
{"id": 1, "title": "Task 1", "isFavorite": false, "status": "active"},
{"id": 2, "title": "Task 2", "isFavorite": true, "status": "done"}
]
''';
```

```
List<dynamic> jsonData = jsonDecode(jsonString);

List<ItemModel> items =
jsonData.map((e) => ItemModel.fromJson(e)).toList();
```

Тепер змінна items містить список об’єктів ItemModel.

Завдання на лабораторну роботу

1. Створити JSON-файл із початковими даними (мінімум 5 елементів).

   Кожен об’єкт повинен містити щонайменше такі поля:

   - id — унікальний ідентифікатор;
   - title — назва елемента;
   - isFavorite / isSaved / isRead — логічне поле;
   - status або інше поле стану.
2. Додати створений JSON-файл у Flutter-проєкт (у папку assets).
3. Додати шлях до JSON-файлу у файл pubspec.yaml, щоб застосунок міг отримувати доступ до цього ресурсу.
4. Реалізувати зчитування JSON-файлу у застосунку та перетворення даних у структури Dart.
5. Створити модель даних (Model class) для опису одного елемента JSON.
6. Реалізувати у моделі методи:
   - fromJson() — для створення об’єкта з JSON;
   - toJson() — для перетворення об’єкта у JSON.
7. Перетворити JSON-масив у список моделей (List<Model>).
8. Відобразити дані у застосунку за допомогою ListView.builder.

9. Реалізувати додавання нового елемента до списку.
10. Реалізувати редагування існуючого елемента (наприклад, зміну назви або статусу).
11. Реалізувати видалення елемента зі списку.
12. Реалізувати перемикання стану елемента залежно від варіанту, наприклад:
    - додати до обраного;
    - позначити як прочитане;
    - позначити як збережене.
13. Реалізувати запис оновлених даних у JSON-файл.
14. Перевірити правильність відображення та оновлення даних у застосунку.

Контрольні запитання

1. Що таке JSON та для чого він використовується у програмуванні?
2. Які основні елементи структури JSON?
3. Які типи даних можуть використовуватися у JSON?
4. Яку бібліотеку використовують у Dart для роботи з JSON?
5. Яка різниця між функціями jsonDecode() та jsonEncode()?
6. У який тип даних перетворюється JSON-об’єкт після декодування?
7. У який тип даних перетворюється JSON-масив після декодування?
8. Як зчитати JSON-файл з папки assets у Flutter?
9. Як записати JSON-дані у файл на пристрої?
10. Для чого використовується пакет path_provider?
11. Яке призначення методу fromJson()?

12. Яке призначення методу toJson()?
13. Як перетворити JSON-масив у список моделей?
14. Які операції можна виконувати з даними, що зберігаються у JSON-файлі?